

CSE416: Software Design Document

Team 3: Yoonseong Kim, Minsu Pak

Yoonseong.kim.1@stonybrook.edu || minsu.pak@stonybrook.edu

Table of Contents

Table of Contents.....	1
1. Introduction.....	4
1.1 Product description.....	4
1.2 Scope.....	6
What This Product Will Do.....	6
What This Will Not Do.....	6
Intended User.....	6
Target Platform.....	6
1.3 Users.....	7
1.4 User feedback.....	8
1.5 Existing alternatives.....	9
1.6 Definitions.....	10
2. Requirements.....	11
2.1 Functional Requirements.....	11
Section 1 - Nav Bar.....	11
Section 2 - Landing Page.....	11
Section 3 - Home/Upload Page.....	11
Section 4 - Results Page.....	12
Section 5 - Pricing Page.....	12
Section 6 - Account Settings Page.....	12
Section 7 - Login Page.....	13
Section 8 - Sign Up Page.....	13
Section 9 - Payment Page.....	14
Section 10 - Features Page.....	14
Section 11 - API Documentation Page.....	14
Section 12 - Stretch Goals.....	15
2.2 Use cases.....	16
Use case 1: Submit Image.....	16
Use case 2: Evaluate Image.....	16
Use case 3: Select Model.....	16
Use case 4: Purchase a Paid Subscription.....	16
Use case 5: Generate API Key.....	17
2.3 Non-functional Requirements.....	18
3. System Architecture.....	19
3.1 Overview.....	19
3.1.1 Front End.....	19
3.1.2 Backend.....	19

3.1.3 Database.....	20
3.2 Data Design.....	21
3.3 UML Sequence Diagrams.....	22
3.4 API Design.....	24
3.5 Deployment.....	26
3.5.1 Front End Deployment.....	26
3.5.2 Back End Deployment.....	26
3.5.3 Database Deployment.....	26
3.5.4 Environment and Configuration Management.....	26
3.6 Code Conventions.....	27
3.6.1 General.....	27
3.6.2 Frontend.....	27
3.6.3 Backend.....	27
4. Schedule.....	29
Milestone 1 - 10/30/2025.....	29
Tasks.....	29
Add API Calls to Login Page.....	29
Add API Calls to Complete Signup Page.....	29
Add API Calls to Complete Analyze Page.....	29
Complete Account Setting Page.....	29
Add API calls to Pricing Page.....	30
Complete Purchase Token Page.....	30
Complete Purchase Page.....	30
Milestone 2 - 11/4/2025.....	30
Tasks.....	30
Login API.....	30
Sign Up API.....	31
Analyze Image Web API.....	31
Analyze Image API API.....	31
Generate API Token API.....	31
Change Password API.....	32
Purchase Plan API.....	32
Purchase Token API.....	32
Cancel Plan API.....	32
Delete Account API.....	32
Train Anime Art Model.....	33
Milestone 3 - 11/11/2025.....	33
Tasks.....	33
Find and Document Issues.....	33
Fix Documented Issues.....	33
Deploy for Beta Release.....	33

Milestone 4 - 11/20/2025.....	34
Tasks.....	34
Implement Stretch Goals.....	34
Get User Feedback.....	34
TaskName.....	34
5. Contributions.....	35
Yoonseong Kim.....	35
Minsu Pak.....	35
Together.....	35

1. Introduction

1.1 Product description

As of 2025, AI tools have been used to create or modify images, and it has been reported that 34 million images are being generated daily (1). With an incredible volume of images created daily, along with the improvement in the quality of the generated image, one may begin to question the authenticity of the image they are viewing. The viewer could have various reasons why they would want the image they are looking at to be a real image made by a human.

1. The viewer may be a customer for a digital image, and they intend to use the digital image commercially. If the image has been made by AI, there is a chance that it could infringe on copyrighted material, and the viewer will be in legal trouble.
2. The viewer is reading a news article, and the article has been augmented with images. The images may have been generated to push a political agenda, and the viewer wants to be grounded.
3. The viewer is browsing artwork, and they want to support real artists and not people who use AI to pretend to be artists.

These are a few reasons why someone would like to know if an image is AI-generated or human-made. Overall, if someone needs to know if an image is AI-generated for any reason, we would like to provide them with a tool to assist in their investigation.

The tool we have decided to provide will be an image classifier trained on AI images and real images to be able to identify minute details that are not perceptible to the human eye. There will be two methods to access our tool; the first method will be through a website to access the classifier. The second method will be for users who are more adept with computers, because it will require them to acquire an API key from us to use our classifying service on their own platform.

3 major features will be implemented into this product. These features will be accessible through the website or our API.

1. The user can give an image to our product, and our product will identify if the image is AI-generated.
 - The image will go to our service, and then our machines will do the work to classify the image. The result of the analysis will be sent back to the user for them to review.
2. The user may select different models to get a better result.
 - There will be various models available to the user. Each model will be fine-tuned to certain styles. Currently, we are offering: general, art, and anime-style.

3. The user may “pay” for an API key to use our classifier on their own service.
 - Our tool will be provided for free through our website for the users. If the user likes our service, then they can have the ability to use our classifiers through an API.

1.2 Scope

What This Product Will Do

This product will provide users with a tool to analyze digital images and determine whether they have been generated by AI or created by humans. The tool will be built on classification models to detect features that distinguish AI-generated images from human ones. The users can access this functionality through a website or by integrating our tool into their own platform using an API we will provide. The product will also support multiple models (general, art, and anime-style) so that users can obtain more accurate results for their specific use case.

What This Will Not Do

The product will not edit, generate, or enhance images. It is only an image classifier. This product also does not guarantee a 100% accurate result, nor will it provide legal advice about copyrighted materials or the image's origin. The product will not moderate or restrict what type of images the user may upload. Additionally, this product will not support pictures of digital images.

Intended User

The primary users of this product include, but are not limited to:

- **Content consumers** who want to check the authenticity of the digital image found online. (e.g., social media posts, news articles, art platforms)..
- **Artists and art communities** who want to check the authenticity of a digital image when purchasing or judging artwork.
- **Businesses** that wish to integrate AI image detection on their platforms via the API (e.g., Art-hosting websites)

Target Platform

The product will be deployed as a web-based application for desktop users. The web-based application can be used on mobile platforms, but the UI will not be designed for mobile users. Additionally, an API service will be available for developers who need AI image classification in their own service.

1.3 Users

There are two primary groups of users for this product:

- **General Users:** This group of users can access the classifier through the website. This group of users may have limited experience with computers, so the website should be simple and intuitive. Their main goal should be to upload an image and then quickly find out if the image was made by an AI or a human. This group can include students, digital art enthusiasts, or general daily web users.
- **Advanced Users:** This group of users is more technically experienced. Their main goal will be to integrate our API into their service, so they will need documentation and an API endpoint. This group will consist of developers from businesses.

1.4 User feedback

Feedback from potential users and stakeholders will be most useful at two stages of development.

1. **UI Prototype Stage:** Once the web interface is implemented, but before the classification model is complete, it would be a useful point to receive feedback. The feedback we will collect at this stage will be on usability, clarity of results, and overall user experience. This information will let us know that the interface we have implemented needs improvement, or that there are elements the user liked. This information can be collected through a closed beta test with testing sessions with classmates and friends.
2. **Post-Development Stage:** After the service goes live, feedback will be focused on the accuracy of the model. The user will have a method to rate the result of the model on the website, and we can track the model's metrics through our service. With the feedback, we can focus on improving models that are not performing as well as they could be.

1.5 Existing alternatives

We have found two existing alternatives:

1. **Sight Engine** (<https://sightengine.com/detect-ai-generated-images>)
2. **Isgen.ai** (<https://isgen.ai/ai-image-detector>)

We found a quality in both of these websites that is both a strength and a weakness. Both websites offer many other services that check for other AI-related content. This makes their service have a lot more features, which can be considered a strength. However, if the focus on the website is to identify images, then the various services make it difficult to navigate back to the image identifier. Therefore, in our product, we will focus only on AI image identification and make it clear on the UI how to navigate back to the identification page.

Additionally, we have taken a look at their pricing page, and we will build our web product to be similar. Since this is a college project and not a real service, we will just pretend the payment has happened. We have also seen that the website does not show information if an image has been scanned on their platform before. Therefore, we will be implementing a method where we can track images and tell users how many times the image has been scanned before.

Furthermore, these existing services do not explicitly offer any AI API's for use. Therefore, our application will be offering this unique feature for users.

1.6 Definitions

- **AI-generated Image:** An image made or modified by artificial intelligence.
- **Classifier:** A machine learning model that analyzes data and then categorizes it into defined classes.
- **API (Application Programming Interface):** A set of functions that allow external software to interact with our system through programming.
- **False Positive/Negative:** A type of classification error.

2. Requirements

These are the requirements of TrueVision.

2.1 Functional Requirements

Section 1 - Nav Bar

- **Req 1-1:** The nav bar shall provide links to navigate
 - **Req 1-1.1:** The nav bar shall provide links to navigate to the home page, features page, pricing page, and account settings page if the user is logged in
 - **Req 1-1.2:** The nav bar shall provide links to navigate to the home page, features page, pricing page, login page, and sign-up page if the user is not logged in

Section 2 - Landing Page

- **Req 2-1:** The landing page shall provide a navbar
- **Req 2-2:** The landing page shall provide a button to direct to the upload page
- **Req 2-3:** The landing page shall provide information about the website and its services

Section 3 - Home/Upload Page

- **Req 3-1:** The results page shall display the image that has been analysed
 - **Req 3-1.1:** The results page shall display the name or the URL of the image file
- **Req 3-2:** The page shall allow users to upload another image to analyse
 - **Req 3-2.1:** The page shall allow users to upload an image from their device
 - **Req 3-2.2:** The page shall allow users to input a URL to an image
- **Req 3-3:** The page shall allow users to click to confirm their image upload and begin analysis
- **Req 3-4:** The home page shall provide a navbar

- **Req 3-5:** The home page shall allow users to select the AI model they would like to use for the image analysis if they are a paid user

Section 4 - Results Page

- **Req 4-1:** The results page shall display the image that has been analysed
 - **Req 4-1.1:** The results page shall display the name or the URL of the image file
- **Req 4-2:** The page shall link back to the home/upload page
- **Req 4-3:** The page shall display whether the analysis believes the image is real or AI-generated
- **Req 4-4:** The page shall display the confidence percentage of the analysed result
- **Req 4-5:** The results page shall display a navbar
- **Req 4-6:** The results page shall allow users to view an AI attention heatmap overlay over the analysed image if they are a paid user
 - **Req 4-6.1:** The results page shall allow users to turn off and on the heatmap overlay

Section 5 - Pricing Page

- **Req 5-1:** The pricing page shall provide a navbar
- **Req 5-2:** The pricing page shall provide buttons to purchase a paid subscription
 - **Req 5-2.1:** If the user is logged in, the button shall redirect to the payment page
 - **Req 5-2.2:** If the user is not logged in, the button shall redirect to the login page
- **Req 5-3:** The pricing page shall provide information about the paid subscription and its services

Section 6 - Account Settings Page

- **Req 6-1:** The account settings page shall display the user's email linked to their account
- **Req 6-2:** The account settings page shall allow users to delete their accounts

- **Req 6-3:** The account settings page shall allow users to save any changes to their account
- **Req 6-4:** The account settings page shall allow users to change their password
 - **Req 6-4.1:** The page shall require users to input their current password to change their password
 - **Req 6-4.2:** The page shall allow users to input their new password
 - **Req 6-4.3:** The page shall allow users to confirm their password change to change their account password
- **Req 6-5:** The page shall display the user's current pricing plan
 - **Req 6-5.1:** The page shall allow users to change their current pricing plan by directing them to the payment page
- **Req 6-6:** The page shall allow users to generate their API key

Section 7 - Login Page

- **Req 7-1:** The login page shall provide a nav bar
- **Req 7-2:** The login page shall provide an area to input the user's account email
- **Req 7-3:** The login page shall provide an area to input the user's password
- **Req 7-4:** The login page shall have a button to submit information to log in
- **Req 7-5:** The login page shall provide a link to the 'sign up page'
- **Req 7-6:** The login page shall check if the email and password combo is valid
- **Req 7-7:** The login page shall redirect to the home page if the login was successful

Section 8 - Sign Up Page

- **Req 8-1:** The sign-up page shall provide a nav bar
- **Req 8-2:** The sign-up page shall provide the user with a method to enter their email address
- **Req 8-3:** The sign-up page shall provide the user with a method to enter their desired password
- **Req 8-4:** The sign-up page shall provide the user with a method to enter their password again to confirm they typed it correctly

- **Req 8-5:** The sign-up page shall provide the user with a method to read the terms and conditions
- **Req 8-6:** The sign-up page shall provide a link to the login page
- **Req 8-7:** The sign-up page shall check if the email is unique
 - **Req 8-7.1:** If unique, the login page shall create the user account.
 - **Req 8-7.2:** If not unique, the login page shall notify the user of the issue.
- **Req 8-8:** The sign-up page shall provide a button to submit the sign-up details entered
- **Req 8-9:** The sign-up page shall redirect to the login page upon signing up
- **Req 8-10.0:** The sign-up page shall provide confirmation that the account has been made if all required fields are valid. This includes: email, password, and terms and conditions.
- **Req 8-10.1:** The sign-up page shall provide an alert if any of the required fields are not valid

Section 9 - Payment Page

- **Req 9-1:** The payment page shall provide a nav bar
- **Req 9-2:** The payment page shall provide a button that simulates purchasing

Section 10 - Features Page

- **Req 10-1:** The features page shall provide a nav bar
- **Req 10-2:** The features page shall provide buttons to relevant pages for presented information
- **Req 10-3:** The features page shall provide information regarding

Section 11 - API Documentation Page

- **Req 11-1:** The documentation page shall provide a nav bar
- **Req 11-2:** The documentation page shall provide a tab to navigate to sections of the documentation
- **Req 11-3:** The documentation page shall provide a back-to-top button
- **Req 11-4:** The documentation page shall provide a way to navigate to the page to get an API key

- **Req 11-4.1:** If logged in, redirect to account settings page
- **Req 11-4.1:** If not logged in, redirect to the login page

Section 12 - Stretch Goals

- Labeling and tracking uploaded images to give statistics to the user
- Allow image links to be used instead of uploading an image

2.2 Use cases

Use case 1: Submit Image

Actor: User

Priority: Essential

Primary Scenario:

1. The user clicks on “select file”
2. The user selects a file
3. The user clicks on upload
4. The file is sent to the backend

Extension Scenario:

2a. The user selects a file that is not an image, and the website rejects it.

Use case 2: Evaluate Image

Actor: Backend

Secondary Actor: AI model

Priority: Essential

Primary Scenario:

1. The backend receives a file
2. The backend checks if the file is an image
3. The backend preprocesses the image
4. The backend passes the image through an AI model
5. The backend sends the result back to the user

Extension Scenario:

2a. The backend sends the user an error message

Use case 3: Select Model

Actor: User

Priority: Essential

Primary Scenario:

1. The user gets an option to pick a model
2. The user selects a model
3. The choice is saved on the website

Extension Scenario:

2a. If the user is not logged in, redirect to the login page

Use case 4: Purchase a Paid Subscription

Actor: User

Secondary Actors: Backend, Database

Priority: Expected

Primary Scenario:

1. The user is logged in
2. The user clicks to purchase the paid plan
3. The user completes the steps to purchase the paid plan
4. The backend receives data that a user has purchased a paid plan
5. The backend authenticates the user's account is real and that their account information is valid through the data stored in the database
6. The database information is altered to show that the user is a paid user
7. The user is shown that their purchase was successful

Extension Scenario:

- 1a. If the user is not logged in, redirect to the login page
- 5a. If there are issues with the user's account data, an error is thrown, and the user is notified of the error. The payment and change of account status are cancelled.

Use case 5: Generate API Key**Actor:** User**Secondary Actor:** Backend, Database**Priority:** Nice to have**Primary Scenario:**

1. The user is logged in and has a paid subscription
2. The user navigates to their account settings page
3. The user is presented with information regarding the management of the API key
4. The user clicks a button to get an API key
5. The backend receives data that the user wants an API key
6. The backend authenticates the user's account is real and that their account information is valid through the data stored in the database
7. The backend checks that the user is a paid user through the database information
8. The backend checks if the account already has an associated API key
9. If an API key already exists, it sends the key information to the website
10. If an API key does not exist, a key is generated, and the key information is sent to the website
11. When the key information is received, the user is shown their API key

Extension Scenario:

- 3a. If the user is not a paid user, then no API key information is shown, and instead, there is a button to prompt the user to purchase a subscription
- 6a. If there are issues with the user's account data, an error is thrown, and the user is notified of the error. The payment and change of account status are cancelled.

7a. If the database information shows the user is not a paid user, but they were able to access the API key management features, this is a sign of a major error, and the user is shown that there is an issue with their account, and an API key is not available

2.3 Non-functional Requirements

Category	NFR	Priority
Performance	Each image shall not take longer than 15 seconds to evaluate	High
	Each model shall have an F1 score > 0.85	High
Cost	There shall be no financial need for any part of the project	High
Security	The website shall use HTTPS	High
	Raw passwords shall not be stored in the database; they must be hashed	High
	Logged-in users will be logged out after a period of time and must log in again.	Medium
	User authentication must occur whenever sensitive actions take place (change of password, purchase of subscription, etc.)	High
Maintainability	Maintenance downtime should be limited to under 1 hour	Low
	Major implementations shall be modularized and follow a predetermined interface	High
Reliability	The service shall be available to the user unless notified of updates and/or maintenance.	Medium

3. System Architecture

In this section, all relevant information on how the project will be made and deployed can be found.

3.1 Overview

For the TrueVision project, many components are working together. Simplified, here is a list of components required to bring TrueVision to life:

- Front End
- Back End
- Database

3.1.1 Front End

The front end of TrueVision serves as the main interface that users will interact with the system. It is responsible for providing a smooth and intuitive experience for both general users and advanced users accessing the API documentation. The front end will be made using **React**, a JavaScript library that enables a streamlined development process by combining HTML and JavaScript into one file. Additionally, Vite will be used to accelerate build time, so development of the front end can be optimized. Lastly, the code will be using **TypeScript** to ensure there is type safety and that runtime errors are reduced.

For styling, **CSS** and **Bootstrap 5** will be used to ensure our design remains consistent across pages. Bootstrap's built-in grid system and premade components will be utilized to make the layout adaptable to different screen sizes. However, the target platform is desktop browsers; therefore, screen sizes that belong to mobile devices will not be as polished. Navigation between pages, such as the homepage, pricing page, and so on, will be handled using **React Router Dom** for transitions without full page reloads.

The front end will communicate with the back end using **HTTPS** requests using the built-in **fetch** feature in TypeScript. This will allow users to upload images to receive classification results, manage their accounts, or use our API by interacting with the back end's **RESTful API**.

3.1.2 Backend

The back end of TrueVision is responsible for managing user requests, running AI image classification models, handling database operations, and maintaining secure communication. It is implemented using **Python 3.11.0** and **FastAPI** for asynchronous

operations and automatic documentation. The automatic documentation is generated via **Swagger UI**, and it makes it convenient for testing.

The back end is structured into separate modules for maintainability. The routes package we created contains API route definitions for core functionalities. These functions can include, but are not limited to:

- User Authentication
- Database operations
- Classification prediction

The classifier component of our back end uses PyTorch and the **Hugging Face Transformers** library. The transformers library is used to load a **Vision Transformer** and fine-tuned weights. Uploaded images are preprocessed using **OpenCV**, **Pillow (PIL)**, and **Torchvision** transforms to standardize input before being passed to the model. The model outputs probabilities indicating whether the uploaded image was AI-generated or human-made, which are then returned to the user as a **JSON** response.

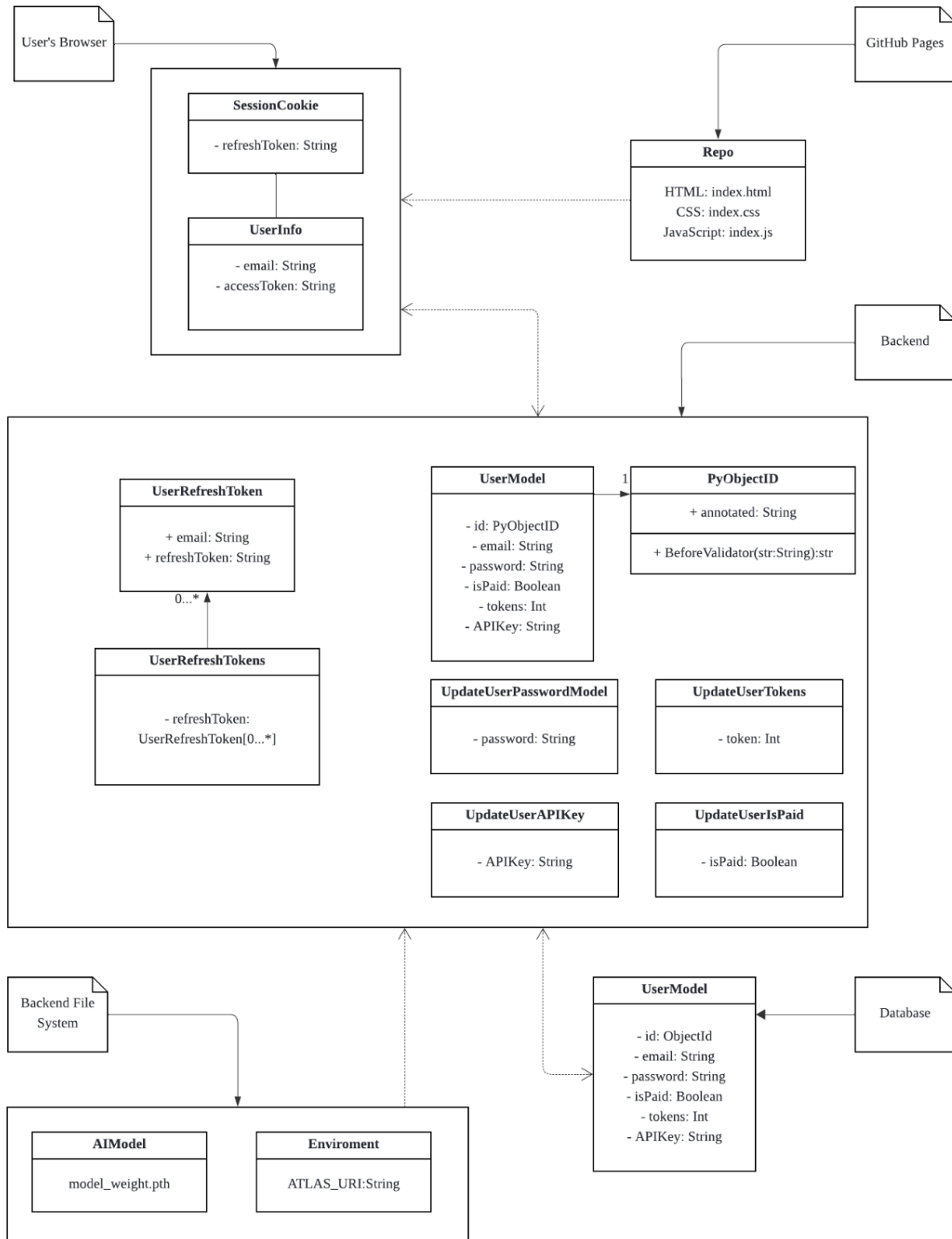
The backend communicates with the MongoDB Atlas database using MongoDB's **PyMongo** driver and its **AsyncMongoClient** function to interface with FastAPI's asynchronous functionality. Additionally, the **PyDantic** library is used to parse between the BSON format used by the database and the JSON format used by FastAPI, and for type validation between MongoDB and Python.

The back end also utilizes **FastAPI's CORSMiddleware** to enable communication between the front end and the **FastAPI** back end. Sensitive back end configuration data, such as database URIs and API keys, is stored separately in environment variables. Additionally, the back end shall utilize **Uvicorn 0.37.0** along with **OpenSSL** to enable **HTTPS**.

3.1.3 Database

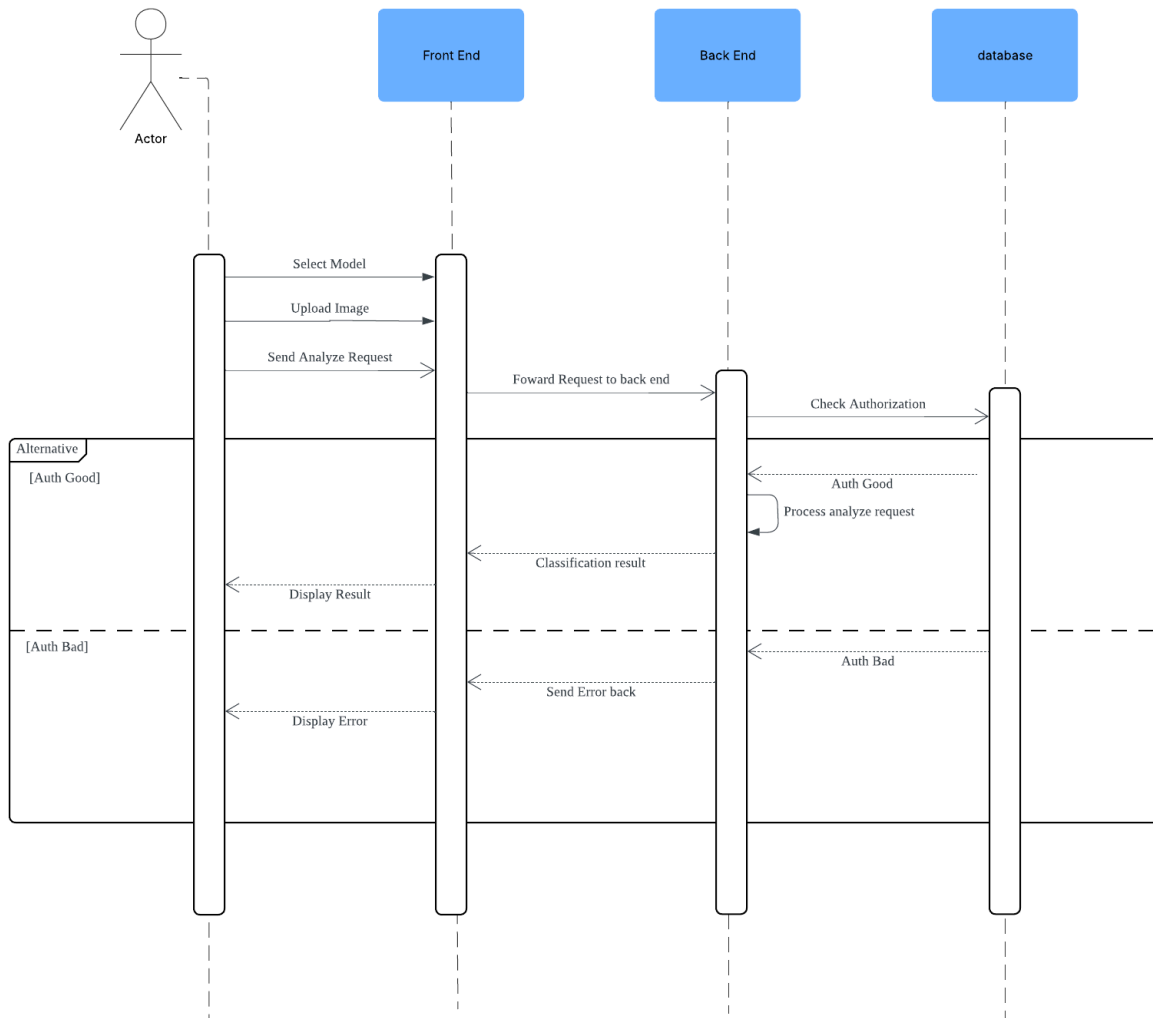
The database for TrueVision uses **MongoDB** hosted on the **MongoDB Atlas** cloud service. It is responsible for securely storing the persistent data required by our service, and handles **CRUD** operations to and from the backend server. MongoDB was initially chosen for its non-relational features and flexibility of schemas that were best suited for our first project idea. This database model was retained for our new project as such features were still useful to implement. Currently, the database is used to store user information for user operations on the front end.

3.2 Data Design

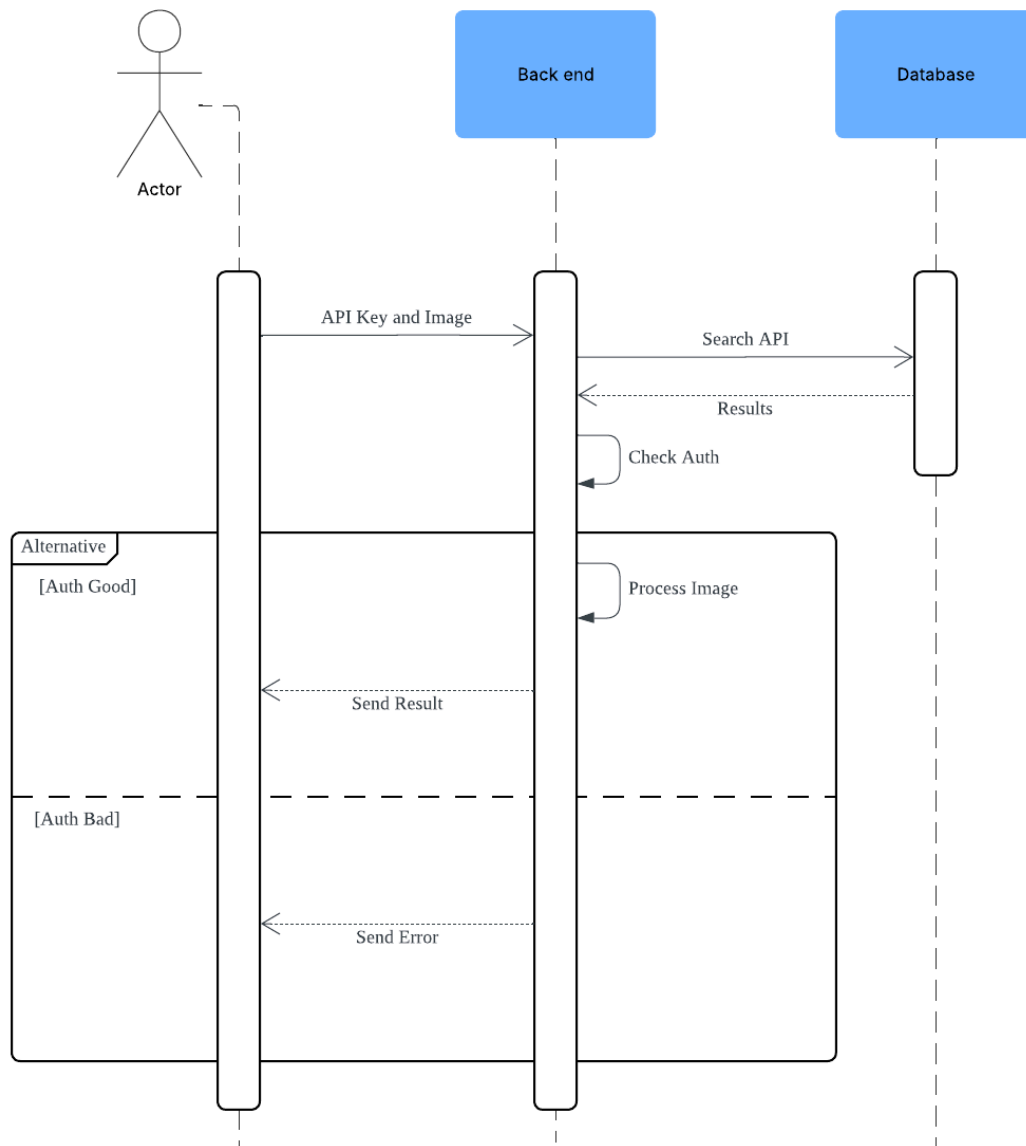


3.3 UML Sequence Diagrams

User Uses Analyze Image Feature



Advanced User API Call



3.4 API Design

Main URL: <https://220.126.157.140:10000>

Name	HTTP Method	URL	Input	Output
Login	POST	/login	String: Email String: HashedPassword	Boolean: Result String: AccessToken Cookie: RefreshToken Status: issues
Sign Up	POST	/sign_up	String: Email String: HashedPassword	Boolean: Result Status: issues
Analyze Image Web	POST	/api/analyze_image_web	String: UserID File: Image	Int: Classification Float: Confidence B64 String: OriginalImage B64 String:HeatMap B64 String: Masked
Analyze Image API	POST	/api/analyze_image_api	String: UserID File: Image String: APIKey	Int: Classification Float: Confidence B64 String: OriginalImage B64 String:HeatMap B64 String: Masked
Generate API Key	POST	/api/generate_key	String: UserID String: AccessToken	Boolean: Result Status: issues
Change Password	POST	/change_password	String: UserID String: AccessToken String: Old PasswordHash String: New PasswordHash	Status: issues
Purchase Plan	POST	/purchase_plan	String: UserID String: AccessToken	Boolean: Result Status: issues
Purchase Token	POST	/purchase_token	String: UserID String: AccessToken Float: TokenAmount	Boolean: Result Status: issues
Cancel Plan	POST	/cancel_plan	String: UserID	Boolean: Result

			String: AccessToken	Status: issues
Delete Account	POST	/delete_account	String: UserID String: AccessToken	Boolean: Result Status: issues
Refresh User Authorization	GET	/refresh	Cookie: RefreshToken	Status: issues String: AccessToken String: UserID
Log Out	GET	/logout	Cookie: RefreshToken	Status: issues

3.5 Deployment

Deployment for TrueVision is designed to be kept simple and functional for demonstration purposes. Since this is a university project, and our team does not have the funds to utilize industry tools, we have looked for free methods of deployment. The system will be deployed as a web application consisting of a front end, a back end, and a database.

3.5.1 Front End Deployment

The front end will be hosted using **GitHub Pages**. It is free and provides a way to reliably host a static site. Using this method is convenient because we are already using **GitHub** to version control our project. Therefore, we only needed to create a branch of our main project and push the compiled **React** project. TrueVision will be accessible via the **GitHub Pages** subdomain assigned to the repository (<https://yoonseongkim2000.github.io/CSE-416-Team3/>).

3.5.2 Back End Deployment

The back-end server will be deployed on a self-hosted environment, running directly on a desktop machine we own. The server uses **Uvicorn 0.37.0** as the ASGI server for running the FastAPI application. The backend operates over **HTTPS** using a self-signed SSL certificate generated with **OpenSSL** on a **WSL (Ubuntu)** environment. This setup ensures **HTTPS** communication between the frontend and backend during data transfers, particularly when users perform authentication actions. If this were a real production project, then a domain could be purchased, and then a legitimate certificate from a CA could be acquired.

The machine hosting the backend also contains the **Vision Transformer (ViT)** model weights, which are loaded locally at runtime by the FastAPI service. This allows the image classification functionality to run entirely on the host machine without requiring external compute services or additional network dependencies.

3.5.3 Database Deployment

The database will be deployed using the **MongoDB Atlas** cloud service. The service is being used with a free account, and provides enough storage and performance to run our website's database operations. The actual server is being run by 3 **AWS** instances in Seoul.

3.5.4 Environment and Configuration Management

TrueVision has some sensitive credentials, such as database connection strings and SSL certificates. These are stored offline and locally with the backend. The environment

variables are defined in a local **“.env”** file. Additionally, the SSL certificate and key, along with the model weights, are stored locally. Relevant information is loaded onto the server upon boot and is not accessible through our back end.

3.6 Code Conventions

This project does not follow a set coding convention; rather, we have developed a system that makes collaboration possible. Here, you can see our short but practical set of conventions for readability and consistency.

3.6.1 General

- Use meaningful and descriptive names. We prefer understanding what something is over brevity.
- Keep functions small and modularized where possible.
- Secrets must never be in the code or committed to GitHub. Use environments.
- Add comments to functions that will be used by others.
- Develop locally and push to pages and the repository when significant progress is made.
- Leave notes in the repository about key functions.

3.6.2 Frontend

- Files and Components
 - Components will use PascalCase
 - Hooks will use camelCase
- Identifiers
 - State variables and handlers will use camelCase
 - Constants will use camelCase
- Styling
 - CSS class names are kebab-case, custom CSS or overrides will use PascalCase
 - CSS IDs are snake_case
 - Make a CSS file for each page
- Formatting
 - Use tabs to signify the depth of TypeScript or HTML
- Type Safety
 - Use TypeScript types and interfaces for component props and functions
 - Avoid using “any” where possible
- Accessibility
 - Attempt to provide alt text

3.6.3 Backend

- Files and Modules
 - Files and modules are snake_case
 - Constants are SCREAMING_SNAKE_CASE

- API route URLs are in snake_case
- Identifiers
 - Functions and variables use snake_case
 - Classes follow PascalCase
- Types
 - When possible, include function input types
- AI/Model Code
 - Keep models global and do not change the device or use model.train() anywhere
 - Use a GPU when possible

3.6.4 Database

- The database follows the generally accepted MongoDB naming conventions. This includes:
 - Collections in snake_case
 - Fields in camelCase
 - Indexes in snake_case

4. Schedule

This is the current work schedule projection for TrueVision.

Monday	Tuesday	Wednesday	Thursday	Friday	Saturday	Sunday
27	28	29	30 Milestone 1	31	1	2
3	4 Milestone 2	5	6	7	8	9
10	11 Milestone 3	12	13	14	15	16
17	18	19	20 Milestone 4: Beta Release	21	22	23

Milestone 1 - 10/30/2025

Goal: Complete implementing the static front end.

Tasks

Add API Calls to Login Page

Level: Functional

Assigned To: Minsu Pak

Dependency: None

Priority: Normal

Description: Complete the web view for the login page and prepare the API calls to make the front end functional.

Add API Calls to Complete Signup Page

Level: Functional

Assigned To: Minsu Pak

Dependency: None

Priority: Normal

Description: Complete the web view for the signup page and prepare the API calls to make the front end functional.

Add API Calls to Complete Analyze Page

Level: Functional

Assigned To: Yoonseong Kim

Dependency: None

Priority: Normal

Description: Complete the web view for the analyze page and prepare the API calls to make the front end functional.

Complete Account Setting Page

Level: Functional

Assigned To: Minsu Pak

Dependency: None

Priority: Normal

Description: Create and complete the web view for the Account settings page. Prepare the API calls to make the front end functional.

Add API calls to Pricing Page

Level: Functional

Assigned To: Minsu Pak

Dependency: None

Priority: Normal

Description: Complete web view for the pricing page and prepare API calls to make the front end functional.

Complete Purchase Token Page

Level: Functional

Assigned To: Yoonseong Kim

Dependency: None

Priority: Normal

Description: Complete web view for the purchase token page and prepare API calls to make the front end functional.

Complete Purchase Page

Level: Functional

Assigned To: Minsu Pak

Dependency: {Add API calls to Pricing Page && Complete Purchase Token Page}

Priority: Low

Description: Create and complete the web view for the purchase page and prepare API calls to make the front end functional.

Milestone 2 - 11/4/2025

Goal: Complete the back end code. Add all relevant functions and APIs required.

Tasks

Login API

Level: Use Case

Assigned To: Minsu Pak

Dependency: None

Priority: High

Description: Make the function to check credentials with database.

Sign Up API

Level: Use Case

Assigned To: Minsu Pak

Dependency: None

Priority: High

Description: Make the function to check and add credentials to database.

Analyze Image Web API

Level: Use Case

Assigned To: Yoonseong Kim

Dependency: None

Priority: High

Description: Create the function to scan the input images and output the analysis response.

Analyze Image API API

Level: Use Case

Assigned To: Yoonseong Kim

Dependency: None

Priority: High

Description: Create the function to check API key and then scan the input images and output the analysis response.

Generate API Token API

Level: Use Case

Assigned To: Yoonseong Kim

Dependency: None

Priority: High

Description: This API needs to generate a unique API key and tie it to the proper account.

Change Password API

Level: Use Case

Assigned To: Minsu Pak

Dependency: None

Priority: Normal

Description: Make the function to allow users to change their password. The API shall check credentials before the password change.

Purchase Plan API

Level: Use Case

Assigned To: Minsu Pak

Dependency: None

Priority: Normal

Description: This function shall update the database such that the user has paid tier.

Purchase Token API

Level: Use Case

Assigned To: Minsu Pak

Dependency: None

Priority: Normal

Description: This function shall update how many tokens the user has in the database.

Cancel Plan API

Level: Use Case

Assigned To: Minsu Pak

Dependency: None

Priority: Normal

Description: This function shall allow the user to cancel their subscription.

Delete Account API

Level: Use Case

Assigned To: Minsu Pak

Dependency: None

Priority: Low

Description: This function shall remove the user entry in the database.

Train Anime Art Model

Level: Sub Task

Assigned To: Yoonseong Kim

Dependency: None

Priority: Low

Description: Train another ViT B16 model using an anime image dataset.

Milestone 3 - 11/11/2025

Goal: Test and debug any API issues found in the front end or back end. Once debugging is complete, deploy for beta release.

Tasks

Find and Document Issues

Level: Sub Task

Assigned To: Team

Dependency: None

Priority: High

Description: The team shall test each API call on the front end and investigate if it is working as intended. Then, attempt to, when possible, to put in erroneous input. When unintended behavior is discovered, it shall be documented.

Fix Documented Issues

Level: Sub Task

Assigned To: Team

Dependency: Find and Document Issues

Priority: High

Description: The team shall address any issues found in task “Find and Document Issues”.

Deploy for Beta Release

Level: Sub Task

Assigned To: Yoonseong Kim

Dependency: Fix Documented Issues

Priority: High

Description: Compile the project and deploy the front and back end to live service.

Milestone 4 - 11/20/2025

Goal: Get user feedback. Fix any bugs and implement user requests if possible and the deploy for release.

Tasks

Implement Stretch Goals

Level: Use Case

Assigned To: Team

Dependency: None

Priority: Low

Description: If time permits, implement any stretch goals we had for this project.

Get User Feedback

Level: Sub Task

Assigned To: Team

Dependency: None

Priority: Normal

Description: Find test users and acquire feedback.

Implement Feedback or Fix Bugs

Level: Sub Task

Assigned To: Team

Dependency: Get User Feedback

Priority: High

Description: Implement any user feedback features that is possible in the allotted time. If any bugs were found, they should be fixed as well.

5. Contributions

This section lists what each member did.

Yoonseong Kim

1. Section 3.3 UML Sequence Diagram
2. Section 3.6 Code Conventions

Minsu Pak

1. Section 3.1.2 Backend
2. Section 3.1.3 Database
3. Section 3.5.3 Database Deployment

Together

1. Presentation
2. Section 3.1 Overview
3. Section 3.2 Data Design
4. Section 3.4 API Design
5. Section 3.5 Deployment
6. Section 4 Schedule